Algorithm 計算機演算法
Class 1: Introduction

逢甲大學 人工智慧研究中心
資 訊 工 程 學 系

許懷中

什麼是演算法？

- 使用電腦來解決問題的技術
- 方法論

問題與解答

- 請回答某數字 x 是否在由 n 個數字所組成的串列 S 中
  - $n = 6$, $S = [10, 7, 11, 5, 13, 8]$, $x = 3$
  - 解答？

方法論

- 當n =6, S = [10, 7, 11, 5, 13, 8] 要解答 x = 3 是否在 S 裡面需要用到什麼方法？
- $n = 100$, $n = 1{,}000$, $n = 1{,}000{,}000$, $n = \ldots\ldots$
- 用電腦解？How?
- **通用**、可以**逐步執行**以**解決問題**的程序

描述一個演算法

- 問題：請回答某數字 x 是否在由 n 個數字所組成的串列 S 中
- 演算法
  - 由 S 中的第一個項目開始，循序地比較 x 與 S 中的項目，直到找到 x ，若找到 x，答案就是 “是” ，又若當 S 中所有的項目都已經被比對過而仍然沒有找到 x 則答案就是 “否”

描述一個演算法 (cont.)

問題：判斷 x 這個 key 是否位於含有 n 個 key 的陣列 S 中。

輸入（參數）：正整數 n，由 n 個 key 所構成的陣列（索引值由 1 到 n），以及 key x。

輸出：$location$，x 在 S 中的位置（如果 x 不在 S 中，將傳回 0）。

```
void seqsearch ( int n ,
                 const keytype S[ ] ,
                 keytype x ,
                 index& location )
{
   location = 1;
   while ( location <= n && S [location] != x )
      location ++;
   if (location > n)
      location =0;
}
```

描述一個演算法 (cont.)

問題：加總所有在陣列 S 中的 n 個數字。

輸入：正整數 n，索引由 1 到 n 的陣列 S。

輸出：sum，S 中所有數字的總和。

```
number sum (int n, const number S[])
{
    index i;
    number result;

    result = 0;
    for (i = 1; i <= n; i++)
        result = result + S[i];
    return result;
}
```

描述一個演算法 (cont.)

問題：以非遞減的順序對 n 個 key 進行排序。

輸入：正整數 n，含有 n 個 key 的陣列 S（索引由 1 到 n）。

輸出：陣列 S，S 中的 n 個 key 已依非遞減順序排列。

```
void exchangesort (int n, keytype S[])
{
    index i, j;
    for (i=1; i<=n; i++)
        for (j=i+1; j<=n; j++)
            if (S[j] < S[i])
                交換 S[i] 與 S[j] 的值；
}
```

描述一個演算法 (cont.)

問題：求得兩個 $n \times n$ 矩陣的乘積。

輸入：正整數 n，二維數字陣列 A 與 B，這兩個矩陣的行索引及列索引都是由 1 到 n。

輸出：二維數字陣列 C，這個矩陣的行索引及列索引都是由 1 到 n，並含有 A 與 B 的乘積。

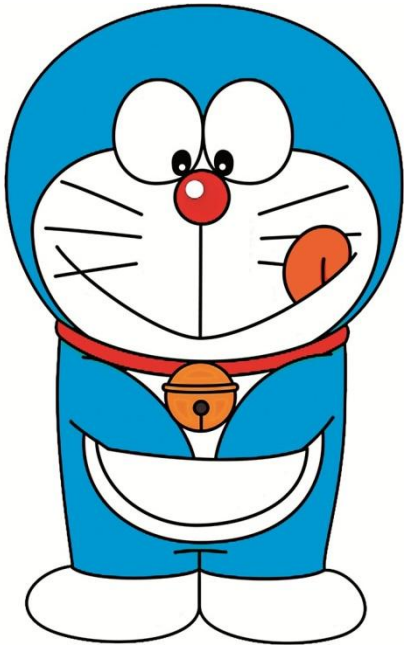
```
void matrixmult (int n,
                 const number A[][] ,
                 const number B[][] ,
                 number C[][])
{
    index i, j, k;

    for (i=1; i<=n; i++)
        for (j=1; j<=n; j++){
            C[i][j] = 0;
            for (k=1; k<=n; k++)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
}
```

Introduction to Algorithms

演算法的效率

演算法的效率

- 為什麼效率很重要？
  - 現代的小叮噹回到了二十世紀末九零年代，他必須要解開一個離散的難題來拯救世界，你希望小叮噹從四次元口袋中拿出什麼東西來拯救世界？
    - 現代的新電腦
      - 搭配 90 年代的演算法
    - 現代的新演算法
      - 搭配 90 年代的電腦



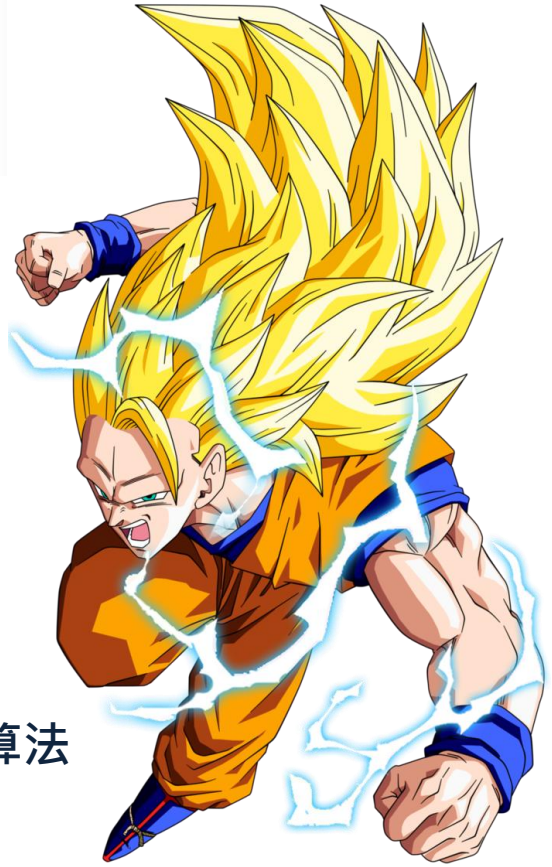
演算法的效率 (cont.)

- 假設 90年代的電腦搭配當年的演算法其戰鬥力為 1
- 現代的電腦搭配 90年代的演算法，其戰鬥力大約為 8,000
- 現代的演算法搭配 90年代的電腦，其戰鬥力大約為 1,000,000

90年代的電腦+ 90年代的演算法

現代的電腦+ 90年代的演算法

90年代的電腦+現代的演算法



現代的電腦+現代的演算法

效率比較高的演算法

問題：判斷 x 這個 key 是否位於含有 n 個 key 的已排序陣列 S 中。

輸入：正整數 n，由 n 個 key 所構成的已排序陣列（依非遞減順序排列，索引值由 1 到 n），以及 x 這個 key。

輸出：$location$，x 在 S 中的位置（如果 x 不在 S 中，將傳回 0）。

效率比較高的演算法 (cont.)

```
void binsearch (int n,
                const keytype S[],
                keytype x,
                index& location)

{
   index low, high ,mid;

   low = 1; high = n;
   location = 0;
   while (low <= high && location == 0) {
      mid = ⌊(low+hight)/2⌋;
      if (x == S[mid])
         location = mid;
      else if (x < S[mid])
         high = mid - 1;
      else
         low = mid + 1;
   }
}
```

- 先比較 x 與陣列的中間項。若這兩數相等，演算法就執行完畢
- 若 x 比中間項小，那麼 x 一定位於陣列的前半段，因此對陣列的前半段重複剛才的搜尋
- 如果 x 比陣列的中間項大，對陣列的後半段重複剛才的搜尋
- 重複執行直到找到 x，或確定x不在陣列中才會停止

效率比較高的演算法 (cont.)

- 表 1.1　當 x 大於所有陣列中的項目時，循序搜尋法與二元搜尋法各自使用的比較次數

陣列大小	循序搜尋法使用的比較次數	二元搜尋法使用的比較次數
128	128	8
1,024	1,024	11
1,048,576	1,048,576	21
4,294,967,296	4,294,967,296	33

費布那西數列

- 遞迴式定義
  - $fib_0 = 0$
  - $fib_1 = 1$
  - $fib_n = fib_{n-1} + fib_{n-2}$ for $n >= 2$
- 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……

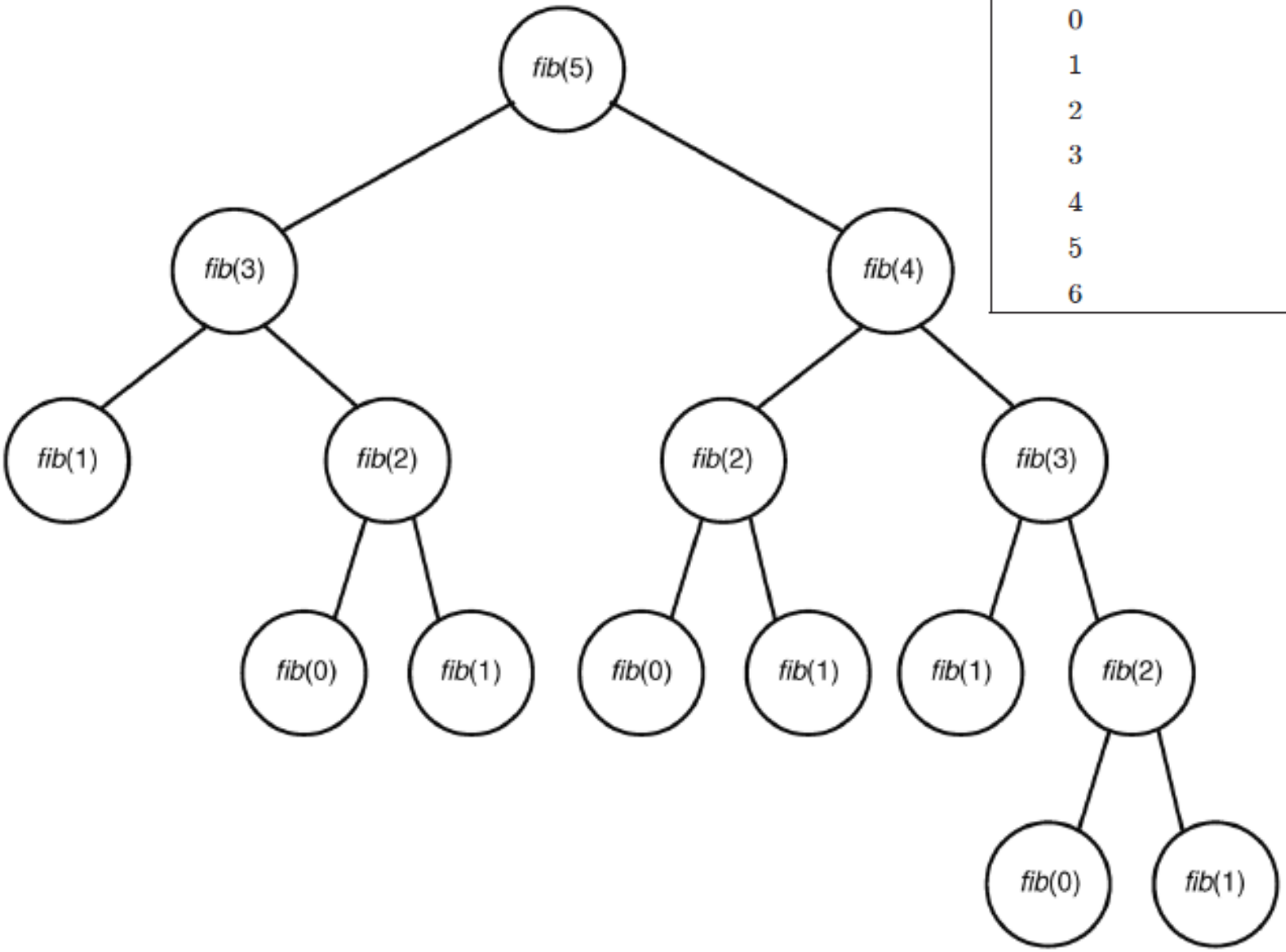
用遞迴解費氏數列

問題：求得費布那西數列的第 n 項。

輸入：非負正整數 n。

輸出：*fib*，費布那西數列的第 n 項。

```
int fib (int n)
{
    if (n <= 1)
        return n;
    else
        return fib(n-1) + fib(n-2);
}
```



n	計算的項目數
0	1
1	1
2	3
3	5
4	9
5	15
6	25

圖 1.2　用演算法 1.6 計算費布那西數列的第五項時，相對應產生的遞迴樹。

用遞迴解費氏數列的效率問題

- 若 $T(n)$ 代表用遞迴解 fib_n 時所需要計算的項目數，假設 $n = 2^k$

$$
\begin{aligned}
T(n) &> 2 \times T(n-2) \\
&> 2 \times 2 \times T(n-4) \\
&> 2 \times 2 \times 2 \times T(n-6) \\
&> 2 \times 2 \times 2 \times \ldots \times 2 \times T(0) \\
&> 2^{\frac{n}{2}}
\end{aligned}
$$

用遞迴解費氏數列的效率問題

證明：我們利用在 n 上執行歸納法來證明。

歸納基底：我們需要兩個基礎的例子，因為歸納步驟中需要用到前兩個例子的結果。對於 $n=2$ 及 $n=3$，圖 1.2 中的遞迴樹顯示

$$T(2) = 3 > 2 = 2^{2/2}$$
$$T(3) = 5 > 2.8323 \approx 2^{3/2}$$

歸納假設：產生歸納假設的其中一個方式是，假設對於所有 $m<n$，這個敘述都成立。接著，在歸納步驟中，證明就 n 而言，此表示同樣的敘述也必須是成立的。我們在這個證明中將使用這個方式。假設對於所有的 m，且 $2 \le m < n$

$$T(m) > 2^{m/2}$$

歸納步驟：我們必須證明 $T(n) > 2^{n/2}$。$T(n)$ 的值等於 $T(n-1)$ 與 $T(n-2)$ 的總和加上一個根節點，因此，

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &> 2^{(n-1)/2} + 2^{(n-2)/2} + 1 \qquad \text{（根據歸納假設）} \\ &> 2^{(n-2)/2} + 2^{(n-2)/2} = 2 \times 2^{(\frac{n}{2})-1} = 2^{n/2} \end{aligned}$$

用迴圈解費氏數列

問題：求得費布那西數列的第 n 項。

輸入：非負正整數。

輸出：fib2，費布那西數列的第 n 項。

```
int fib2 (int n)
{
    index i;
    int f[0..n];
    f[0]=0;
    if (n > 0)
        f[1]=1;
        for (i=2; i<=n; i++)
            f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

動態規劃

比較演算法的效率

n	$n+1$	$2^{n/2}$	動態規劃算法 需要的時間	純遞迴算法 計算時間之下限
40	41	1,048,576	41 奈秒 *	1048 微秒 †
60	61	1.1×10^{9}	61 奈秒	1 秒
80	81	1.1×10^{12}	81 奈秒	18 分
100	101	1.1×10^{15}	101 奈秒	13 天
120	121	1.2×10^{18}	121 奈秒	36 年
160	161	1.2×10^{24}	161 奈秒	3.8×10^{7} 年
200	201	1.3×10^{30}	201 奈秒	4×10^{13} 年

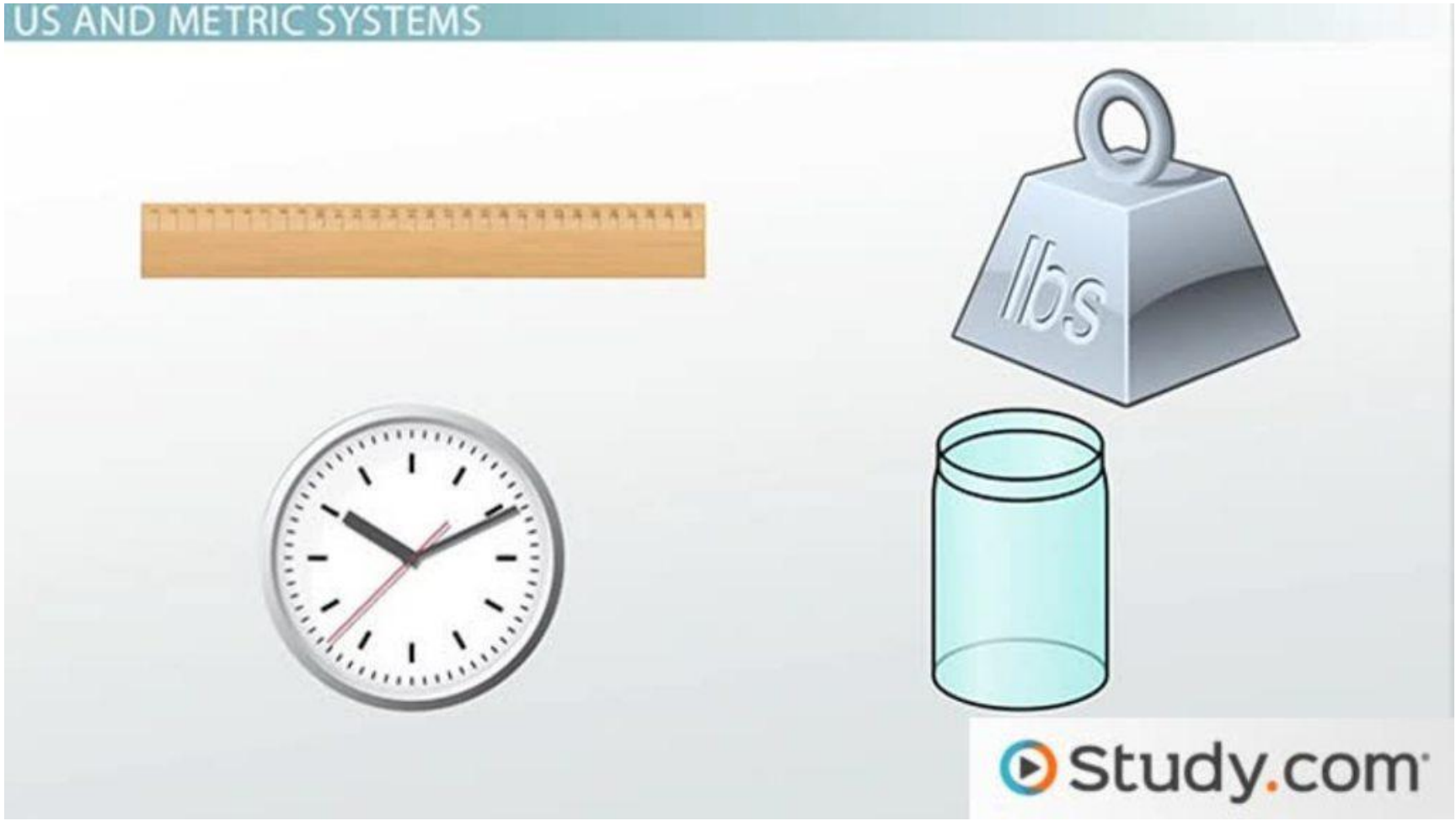
*1 奈秒 $=10^{-9}$ 秒。

†1 微秒 $=10^{-6}$ 秒。

Introduction to Algorithms

分析演算法

用什麼來衡量？



衡量演算法效率的單位

- 輸入大小
- 基本運算執行的次數
- 什麼東西被忽略了？Why？

輸入大小

- 下列問題的輸入大小為？
  - 判斷 x 這個 key 是否位於含有 n 個 key 的陣列中
  - 加總所有在陣列 S 中的 n 個數字
  - 求兩個 $n \times n$ 矩陣的乘積
  - 求得費氏數列的第 n 項
  - 回答 n 是否為一個質數

基本運算

- 用來解決下列問題之演算法的基本運算為？
  - 判斷 $key\ x$ 是否在一個有 n 個 key 的陣列中
  - 加總所有在陣列 S 中的 n 個數字
  - 求得兩個 $n \times n$ 矩陣的乘積
  - 求得費氏數列的第 n 項

時間複雜度 (Time Complexity)

- 求得每個不同的輸入大小，該演算法所執行的基本運算次數
- **T(n)**: 對於一個大小為n的個體，該演算法所執行的基本運算次數
  - $T(n)$即被稱為演算法之**所有情況的時間複雜度**

所有情況的時間複雜度 (1)

問題：加總所有在陣列 S 中的 n 個數字。

輸入：正整數 n，索引由 1 到 n 的陣列 S。

輸出：sum，S 中所有數字的總和。

```
number sum (int n, const number S[])
{
    index i;
    number result;

    result = 0;
    for (i = 1; i <= n; i++)
        result = result + S[i];
    return result;
}
```

$$T(n) = n$$

所有情況的時間複雜度 (2)

問題：以非遞減的順序對 n 個 key 進行排序。

輸入：正整數 n，含有 n 個 key 的陣列 S（索引由 1 到 n）。

輸出：陣列 S，S 中的 n 個 key 已依非遞減順序排列。

```
void exchangesort (int n, keytype S[])
{
    index i, j;
    for (i=1; i<=n; i++)
        for (j=i+1; j<=n; j++)
            if (S[j] < S[i])
                交換 S[i] 與 S[j] 的值；
}
```

$$T(n) = \frac{n(n-1)}{2}$$

所有情況的時間複雜度 (3)

問題：求得兩個 $n \times n$ 矩陣的乘積。

輸入：正整數 n，二維數字陣列 A 與 B，這兩個矩陣的行索引及列索引都是由 1 到 n。

輸出：二維數字陣列 C，這個矩陣的行索引及列索引都是由 1 到 n，並含有 A 與 B 的乘積。

```
void matrixmult (int n,
                 const number A[][] ,
                 const number B[][] ,
                 number C[][])
{
    index i, j, k;

    for (i=1; i<=n; i++)
        for (j=1; j<=n; j++){
            C[i][j] = 0;
            for (k=1; k<=n; k++)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
}
```

$$T(n) = n^3$$

所有情況的時間複雜度 (4)

問題：判斷 x 這個 key 是否位於含有 n 個 key 的陣列 S 中。

輸入（參數）：正整數 n，由 n 個 key 所構成的陣列（索引值由 1 到 n），以及 key x。

輸出：$location$，x 在 S 中的位置（如果 x 不在 S 中，將傳回 0）。

```
void seqsearch ( int n ,
                 const keytype S[ ] ,
                 keytype x ,
                 index& location )
{
   location = 1;
   while ( location <= n && S [location] != x )
      location ++;
   if (location > n)
      location =0;
}
```

$$T(n) = ???$$

時間複雜度的表示方式

- 所有情況的時間複雜度不僅僅是 n 的函數
  - 最差狀況　　$\mathrm{W}(n)$
  - 平均狀況　　$\mathrm{A}(n)$
  - 最佳狀況　　$\mathrm{B}(n)$

循序演算法的時間複雜度

問題：判斷 x 這個 key 是否位於含有 n 個 key 的陣列 S 中。

輸入（參數）：正整數 n，由 n 個 key 所構成的陣列（索引值由 1 到 n），以及 key x。

輸出：$location$，x 在 S 中的位置（如果 x 不在 S 中，將傳回 0）。

```
void seqsearch ( int n ,
                 const keytype S[ ] ,
                 keytype x ,
                 index& location )
{
   location = 1;
   while ( location <= n && S [location] != x )
      location ++;
   if (location > n)
      location =0;
}
```

$$W(n) = n$$

$$B(n) = 1$$

循序演算法的時間複雜度

- 假設已知 x 在 S 中， S 中各項目是相異的，而 x 出現在 S 中任意位置的機率都相等

$$\mathrm{A}(n) = \frac{(n+1)}{2}$$

- 考慮 x 可能不在 S 中的狀況
  - 假設 x 出現在 S 中的機率為 p

$$\mathrm{A}(n) = \frac{p}{n} \times \frac{n(n+1)}{2} + n(1-p) = n\left(1 - \frac{p}{2}\right) + \frac{p}{2}$$

注意平均這件事情

比克叔叔！

電視新聞說：台灣平均每人財富有665萬元！聽起來很棒耶！

你爸你媽平均有一顆睪丸耶。

複雜度函數

- 任一將正整數映射到非負實數的函數

$$f(n) = n$$

$$f(n) = n^2$$

$$f(n) = \lg n$$

$$f(n) = 3n^2 + 4n$$

演算法分析

- 分析演算法的效率
  - 考慮輸入大小與基本運算用時，忽略了什麼？
  - 假設有一個問題有兩種演算法可解
    - 演算法 1：$T(n) = n^2$ ，基本運算需要 1 秒
    - 演算法 2：$T(n) = n$ ，基本運算需要 1,000 秒
    - 我們要用哪一個？
    - 在 ***n* 夠大**的情況下，不論執行基本運算所費的時間多寡，時間複雜度為 n 的演算法會比另一個時間複雜度為 n^2 的演算法更有效率
- 分析演算法的正確性

Introduction to Algorithms

ORDER (量級)

演算法效率比較

- 假設有兩個演算法
  - 時間複雜度分別為 $0.01n^2$ 與 $100n$
  - 基本運算、額外負擔指令需時皆相仿
  - 哪個演算法比較有效率？比較有效率的條件？

以函數形式表示演算法效率

- 時間複雜度為 n, $10n$, $0.01n$, ...
  → **線性時間演算法**
- 時間複雜度為 n^2, $10n^2$, $0.01n^2$, ...
  → **平方時間演算法**
- **$5n^2$, $5n^2+100$ ...**
  → **純平方函數**
- **$0.1n^2 + n + 100$**
  → **完全平方函數**

n	$0.1n^2$	$0.1n^2+n+100$
10	10	120
20	40	160
50	250	400
100	1,000	1,200
1,000	100,000	101,100

二次項最終主控整個函數的值

複雜度量級 (不嚴謹的講法)

- $0.1n^2 + n + 100 \in \Theta(n^2)$
- 平方時間演算法、$\Theta(n^2)$ 演算法
- 範例
  - 交換排序演算法 $\mathrm{T}(n) = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2} \in \Theta(n^2)$

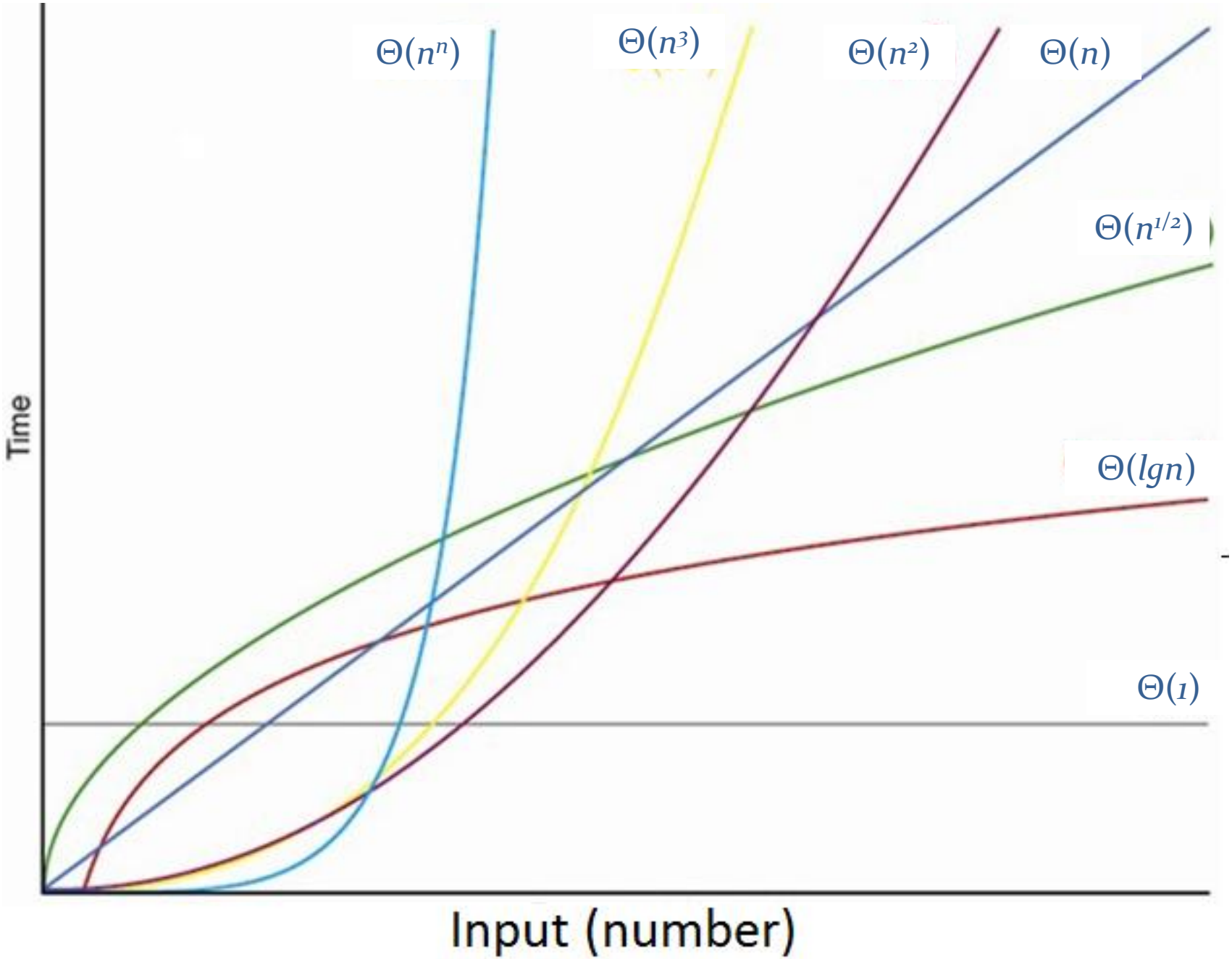
複雜度類別 (cont.)

- 常見的複雜度類別

$\Theta(lgn)$、$\Theta(n)$、$\Theta(nlgn)$、$\Theta(n^2)$、$\Theta(n^3)$、$\Theta(2^n)$

n	$f(n) = \lg n$	$f(n) = n$	$f(n) = n \lg n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.003 微秒 *	0.01 微秒	0.033 微秒	0.10 微秒	1.0 微秒	1 微秒
20	0.004 微秒	0.02 微秒	0.086 微秒	0.40 微秒	8.0 微秒	1 毫秒 †
30	0.005 微秒	0.03 微秒	0.147 微秒	0.90 微秒	27.0 微秒	1 秒
40	0.005 微秒	0.04 微秒	0.213 微秒	1.60 微秒	64.0 微秒	18.3 分
50	0.006 微秒	0.05 微秒	0.282 微秒	2.50 微秒	125.0 微秒	13 天
10^2	0.010 微秒	0.1 微秒	0.664 微秒	10 微秒	1 毫秒	4×10^{13} 年
10^3	0.010 微秒	1 微秒	9.966 微秒	1 毫秒	1 秒	
10^4	0.013 微秒	10 微秒	130 微秒	100. 毫秒	16.70 分	
10^5	0.017 微秒	0.1 毫秒	1.670 毫秒	10 秒	11.6 天	
10^6	0.020 微秒	1.0 毫秒	19.930 毫秒	16.70 分	31.70 年	
10^7	0.023 微秒	0.01 秒	2.660 秒	1.16 天	31,709 年	
10^8	0.027 微秒	0.1 秒	2.660 秒	115.7 天	3.17×10^7 年	
10^9	0.030 微秒	1.00 秒	29.900 秒	31.70 年		

*1 微秒 $=10^{-6}$ 秒。†1 毫秒 $=10^{-3}$ 秒。


$\Theta(n^n)$
$\Theta(n^3)$
$\Theta(n^2)$
$\Theta(n)$
$\Theta(n^{1/2})$
$\Theta(lgn)$
$\Theta(1)$
Time
Input (number)

量級 – Big O

- 給定一複雜度函數 $f(n)$ ，$O(f(n))$就是由一些複雜度函數 $g(n)$構成的集合

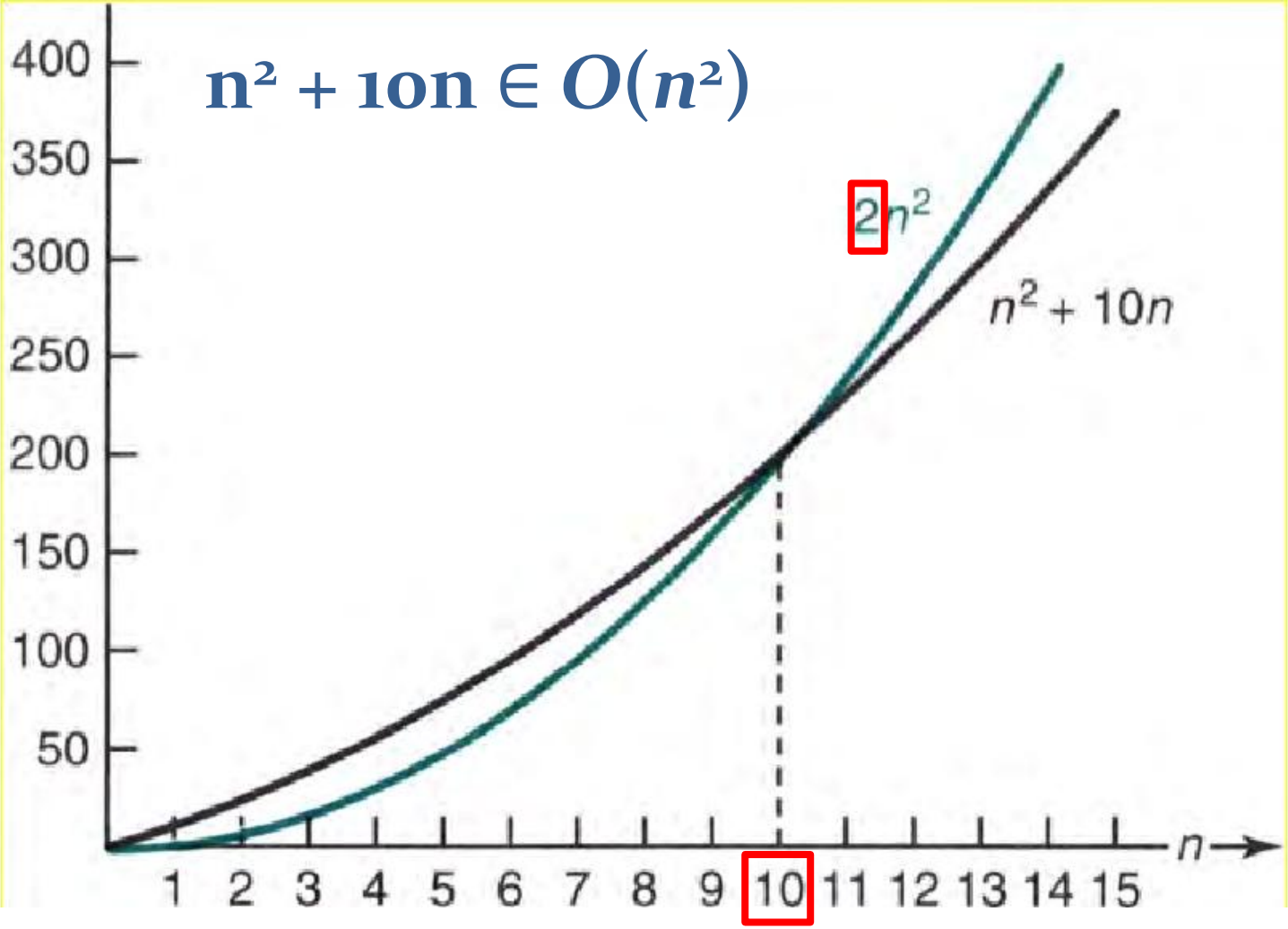
- $g(n) \in O(f(n))$ 則

  $\forall\, g(n)$,

  $\exists$ 某正實數常數 c 與某非負整數 N

  使得 $\forall\, n \geq N,\ g(n) \leq c \times f(n)$

Big O – 範例


$n^2 + 10n \in O(n^2)$
400
350
300
250
200
150
100
50
$2n^2$
$n^2 + 10n$
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
n

Big O

- 交換排序法：$\mathrm{T}(n) = \frac{n(n-1)}{2}$

  當 $n = 0$, $\mathrm{T}(n) \leq \frac{1}{2}n^2$, $\mathrm{T}(n) \in O(n^2)$
- 證明 $5n^2 \leq O(n^2)$

  因為，對於$n \geq 0$，$5n^2 \leq 5n^2$

  我們可以取 $c = 5$ 以及 $N = 0$ 以獲得我們想要的結果
- Big O 在一個函數上設置了漸進**上限 (upper bound)**

Big O (cont.)

- $g(n) \in O(f(n))$ 代表 $g(n)$ 至少跟 $f(n)$ 一樣好
  - 因為當 n 大於某一個常數時，$c^* f(n)$ 總是比 $g(n)$ 大
  - 交換排序法的時間複雜度 $T(n) \in O(n^2)$ 代表著 交換排序法最多只會與某個平方時間演算法消耗差不多的時間(只差一個係數)

Big O 的迷思

- 只會有一組 Big O?
  - $T(n) = \frac{n(n-1)}{2}$, $T(n) \in O(n^2)$
  - $T(n) \in O(n^2+10n)$?
- 每一組 Big O 只會有一組 c 與 N?
  - 以 c=2, N=10 證明 $n^2+10n \in O(n^2)$
  - c = 11, N = 1 可以嗎？

Omega Ω

- 在一個函數上設置一個漸進**下限**
- 給定一複雜度函數 $f(n)$ ， $\mathbf{\Omega}(f(n))$就是由一些複雜度函數$g(n)$構成的集合
- $g(n) \in \mathbf{\Omega}(f(n))$ 則

  $\forall\, g(n),$

  $\exists$ 某正實數常數 c 與某非負整數 N

  使得 $\forall\, n \geq N,\ g(n) \geq c \times f(n)$

Omega Ω (cont.)

- 證明 $5n^2 \in \Omega(n^2)$

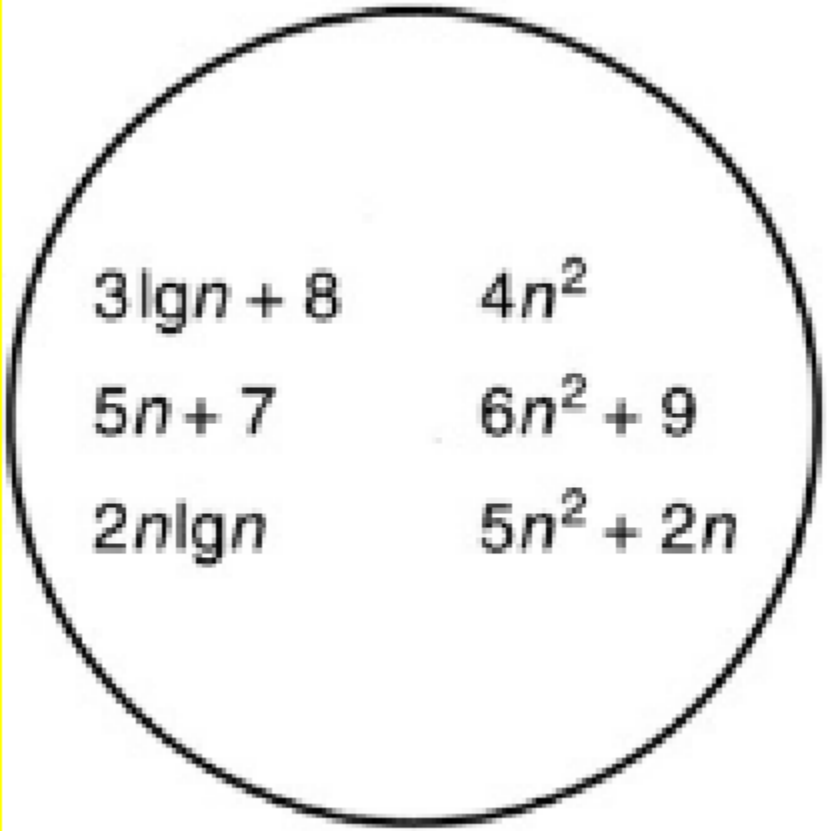
  因為，對於 $n \geq 0$，$5n^2 \geq 1 \times n^2$

  取 $c = 1$ 以及 $N = 0$ 可得證

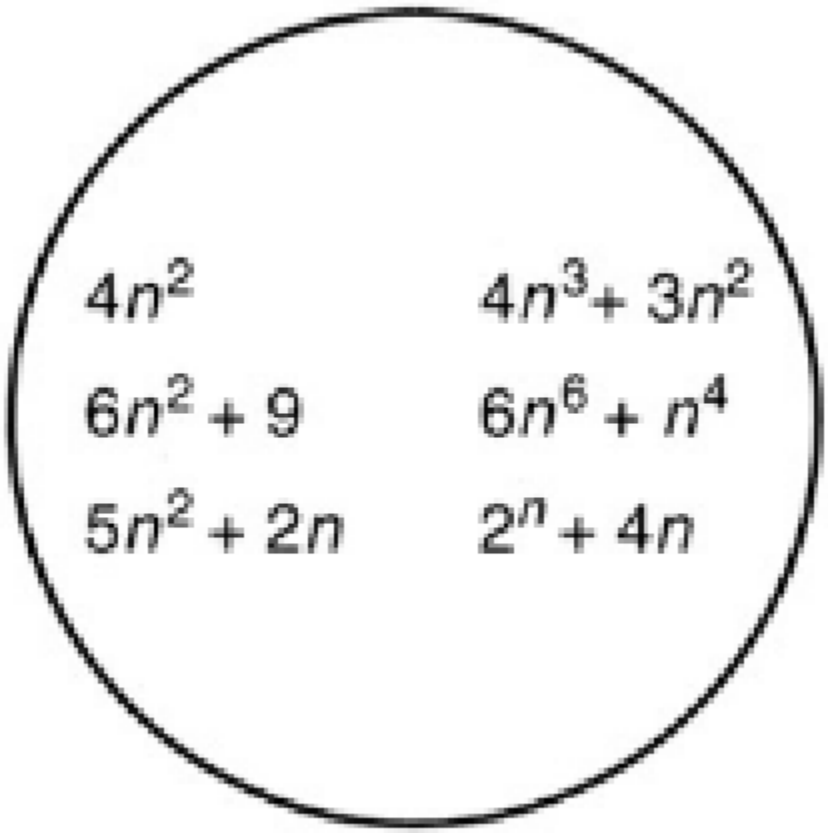
- 交換排序法：$T(n) = \frac{n(n-1)}{2}$

  當 $N = 3$, $T(n) > \frac{1}{4}n^2$, $T(n) \in \Omega(n^2)$

Big O 與 Omega Ω

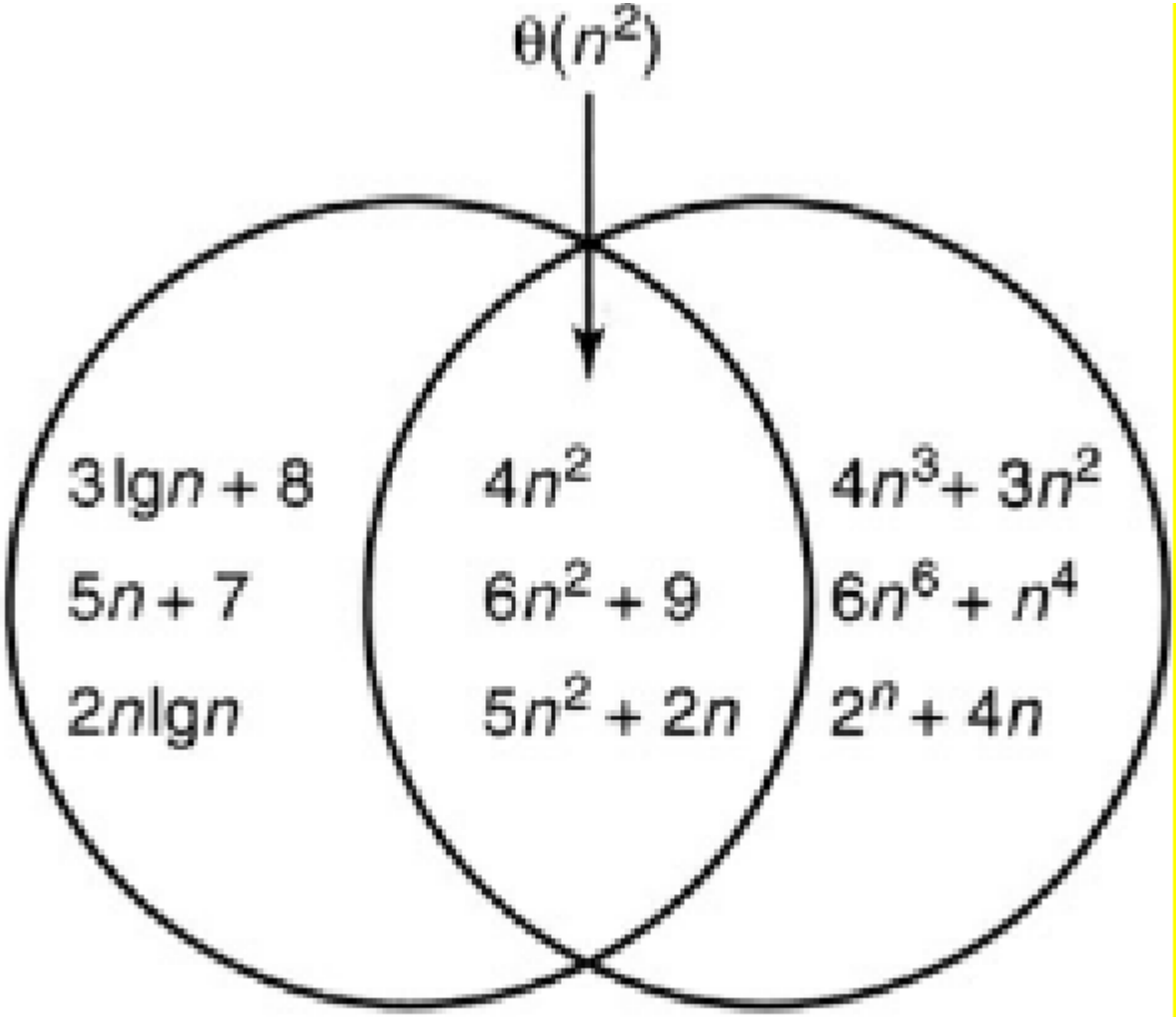


(a) $O(n^2)$



(b) $\Omega(n^2)$

從 Big O 與 Ω 求 Θ



(c) $\theta(n^2) = O(n^2) \cap \Omega(n^2)$

從 Big O 與 Ω 求 Θ (cont.)

定義　對於一個給定的複雜度函數 $f(n)$，

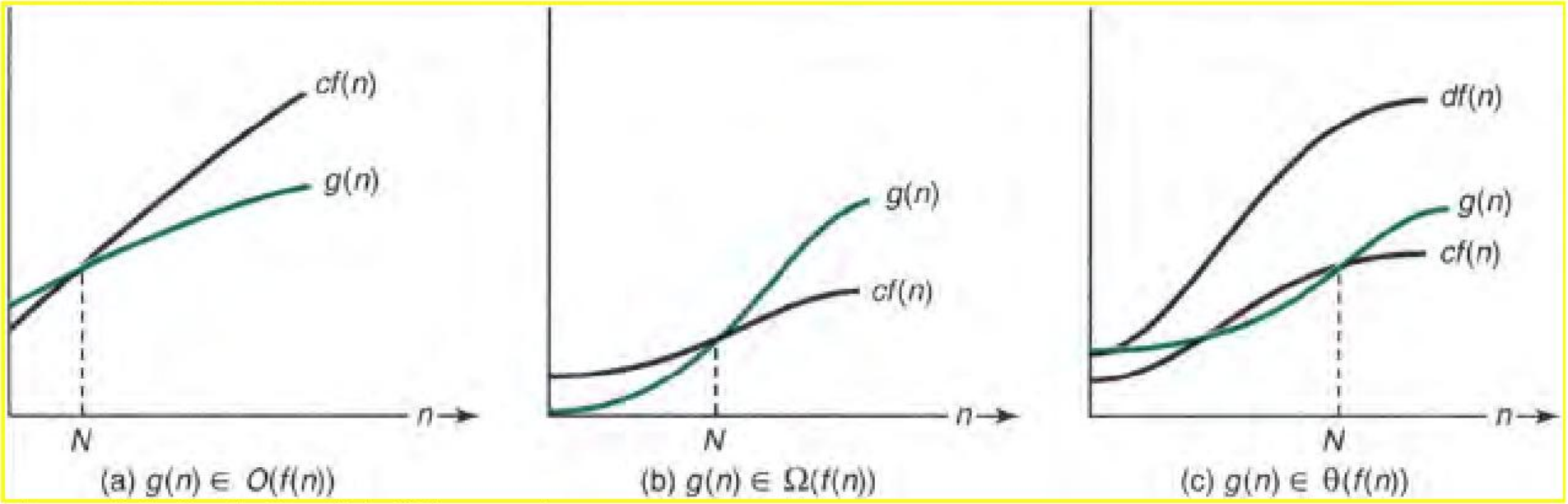
$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

這代表的是 $\Theta(f(n))$ 就是由一些複雜度函數 $g(n)$ 構成的集合。其中，對於每一個 $g(n)$，必存在正實數常數 c 與 d 及某個非負整數 N，使得對於所有 $n \geq N$，

$$c \times f(n) \leq g(n) \leq d \times f(n)$$

若 $g(n) \in \Theta(f(n))$ 則 $g(n)$ 為 $f(n)$ 量級 (Order $f(n)$)

從 Big O 與 Ω 求 Θ (cont.)



從 Big O 與 Ω 求 Θ (cont.)

- 交換排序法：$\mathrm{T}(n) = \frac{n(n-1)}{2}$
  - $\mathrm{T}(n) \in \mathrm{O}(n^2)$ 同時 $\mathrm{T}(n) \in \Omega(n^2)$
  - $\mathrm{T}(n) \in \mathrm{O}(n^2) \cap \Omega(n^2)$
  - $\mathbf{T}(\boldsymbol{n}) \in \boldsymbol{\Theta}(\boldsymbol{n}^2)$
- 交換排序法是平方量級的演算法

想想！

- $B(n)$, $W(n)$, $A(n)$ 與 $\Omega(n)$, $O(n)$, $\Theta(n)$ 有何不同？
- 為什麼需要定義 $\Omega(n)$, $O(n)$, $\Theta(n)$？

如何證明 $g(n)$ 不屬於某一個量級

- 使用反證法 (歸謬) 來證明 n 不在$\Omega(n^2)$中

假設 $n \in \Omega(n^2)$

$\exists$ 某個正值常數 c 以及某個非負整數N，使得

$\forall\, n \geq N$，$n \geq cn^2 \Rightarrow \frac{1}{n} \geq c$

由於 c 為常數, 我們一定找得到一個 $n \geq N$ 使得 $\frac{1}{n} < c$

矛盾，假設有誤 $n \notin \Omega(n^2)$

Small o

- 給定一複雜度函數 $f(n)$ ，$o(f(n))$就是由一些複雜度函數$g(n)$構成的集合
- $g(n) \in o(f(n))$ 則

  $\forall\, g(n)$,

  $\forall$ 正實數常數 c, $\exists$某非負整數 N

  使得 $\forall\, n \geq N$, $g(n) \leq c \times f(n)$

Small o – 範例

- 證明 $n \in o(n^2)$

如果 $n \in o(n^2)$ 則給定任意 $c > 0$, $\exists$ N 使得 $n \leq cn^2$

不等式同除以 cn 可得 $n \geq \frac{1}{c}$

由於 c 為常數, 必定存在一個 N 使得 $\forall\, n \geq N, \frac{1}{n} < c$

上述對任意正實數 c 皆成立

因此可得證 $n \in o(n^2)$

關於量級的一些重要性質

1. $g(n) \in O(f(n))$ 若且唯若 $f(n) \in \Omega(g(n))$
2. $g(n) \in \Theta(f(n))$ 若且唯若 $f(n) \in \Theta\ (g(n))$
3. 若 $b>1$ 且 $a>1$，則 $\log_a n \in \Theta(\log_b n)$

   這表示所有的對數複雜度函數都在同一個複雜度類別中。我們將會用 $\Theta(\lg\ n)$ 來代表這個類別。
4. 若 $b>a>0$，則

$$a^n \in o(b^n)$$

   此表示所有指數函數並不在同一個複雜度類別中。

關於量級的一些重要性質 (cont.)

5. 對於所有的 $a>0$

$$a^n \in o(n!)$$

暗指了 $n!$ 比任一個指數複雜度函數都要差。

6. 考慮下列的複雜度類別的量級：

$$\Theta(\lg n) \quad \Theta(n) \quad \Theta(n \lg n) \quad \Theta(n^2) \quad \Theta(n^j) \quad \Theta(n^k) \quad \Theta(a^n) \quad \Theta(b^n) \quad \Theta(n!)$$

其中 $k>j>2$ 且 $b>a>1$。若一個複雜度函數 $g(n)$ 所在的類別在 $f(n)$ 所在類別的左方，那麼

$$g(n) \in o(f(n))$$

7. 若 $c \geq 0$，$d>0$，$g(n) \in O(f(n))$ 且 $h(n) \in \Theta(f(n))$，那麼

$$c \times g(n) + d \times h(n) \in \Theta(f(n))$$

關於量級的一些重要性質 (cont.)

性質 3 說明所有的對數複雜度函數都在同一個複雜度類別中。例如

$$\Theta(\log_4 n) = \Theta(\lg n)$$

這代表的是 $\log_4 n$ 之於 $\lg n$ 的關係等同於與 $7n^2 + 5n$ 之於 n^2 的關係。

性質 6 說明任一對數函數最終都會比任一多項式函數為佳，任一個多項式函數最終都會比任一指數函數好，而任一指數函數最終都會比任一階乘函數好，例如，

$$\lg n \in o(n), \qquad n^{10} \in o(2^n), \qquad \text{及} \qquad 2^n \in o(n!)$$

關於量級的一些重要性質 (cont.)

性質 6 與 7 可被重複地使用。例如，我們可以用下面的方式，證明 $5n+3\lg n+10n\lg n+7n^2 \in \Theta(n^2)$。重複地應用性質 6 與 7，我們得到

$$7n^2 \in \Theta(n^2)$$

此表示

$$10n\lg n + 7n^2 \in \Theta(n^2)$$

此表示

$$3\lg n + 10n\lg n + 7n^2 \in \Theta(n^2)$$

此表示

$$5n + 3\lg n + 10n\lg n + 7n^2 \in \Theta(n^2)$$

Summary

- 丟棄某演算法之 $T(n)$ 中的低次項，以求得該演算法的量級
- 藉由 $T(n) \in O(f(n))$ 且 $T(n) \in \Omega(f(n))$ 便可以推論出$T(n) \in \Theta(f(n))$
- 在某些情況下，證明 $T(n) \in O(f(n))$極為容易，但是證明 $T(n) \in \Omega(f(n))$ 卻很困難。在這樣的情況下我們只好退而求其次證明 $T(n) \in O(f(n))$
- 有些作者會使用

  $f(n) = \Theta(n^2)$ 來取代 $f(n) \in \Theta(n^2)$

  $f(n) = O(n^2)$ 來取代 $f(n) \in O(n^2)$